

Spring

JWT



JWT



- Json Web Token è un sistema di autenticazione basato su un oggetto di tipo Json.
- Le informazioni vengono memorizzate in una stringa alfanumerica chiamata Token.
- Il token è suddiviso in tre parti.
  - Header – contiene le informazioni di codifica del token
  - PayLoad – contiene i nostri dati
  - Signature – ci permette di controllare se il token è stato modificato
- Possiamo utilizzare JWT per controllare se:
  - Un utente utilizza delle credenziali esatte – autenticazione
  - Un utente ha accesso a delle specifiche risorse – autorizzazione
  - Da quanto tempo l'utente ha effettuato una determinata operazione – Validazione



JWT



- Quello che andremo a fare è creare un server esterno di validazione che ci permetterà di creare il nostro token
- Nella nuova applicazione andiamo ad importare lo starter security e jjwt che ci servirà per creare effettivamente il nostro server

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.13.0</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.13.0</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.13.0</version>
</dependency>
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```



JWT



- Per gestire JWT dobbiamo creare un RestController con tre metodi
  - Il primo per creare un Token
  - Il secondo per refreshare il token
  - Il terzo sarà l'ExceptionHandler



JWT



- Dovremmo anche modificare l'utente facendo implementare UserDetails
- Partiamo da un esempio con due ruoli, un utente e un gestore

```
public enum Ruolo {
    UTENTE("UTENTE"),
    GESTORE("GESTORE");
    private String ruolo;
    Ruolo(String ruolo){
        this.ruolo=ruolo;
    }
    public String getNome(){return ruolo;}
}
```



JWT



```
public class Utente implements UserDetails {

    private Ruolo ruolo;
    private String username;
    private String password;

    //costruttori e metodi della classe

}
```



JWT



- Tra i vari metodi che dovremmo implementare sull'utente ci sarà quello che ci permetterà di vedere i ruoli di quell'utente

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_"+ruolo.getNome()));
}
```



JWT



- Oltre a questo avremo bisogno di:
  - Un service per la gestione del token
  - Una classe di configurazione con i bean di configurazione
  - Un filter di autenticazione ottenuto estendendo «`OncePerRequestFilter`»
  - Una classe di configurazione che ci permetterà di gestire la filterChain
  - La classe che gestirà i Ruoli che hanno accesso (l'utente) dovrà estendere «`UserDetails`»



JWT – service gestione token



- Il serice per la generazione e gestione dei token potremmo idealmente dividerlo in 2 parti, la prima per la generazione, la seconda per la validazione
- Entrambe utilizzeranno una javax.crypto.SecretKey di sicurezza che servirà per validare il token

```
@Service
public class GestoreToken {

    private static final String SECRET_KEY = "chiaveAbbastanzaLunga";


    private SecretKey getSignInKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(SECRET_KEY.getBytes()));
    }

}
```



JWT – service gestione token - claims



- All’interno del nostro token potremo avere quante informazioni vogliamo, tutte queste informazioni faranno parte di un oggetto chiamato Claims che sarà contenuto nel nostro token
- Il token che genereremo sarà inserito nell’header, oltre ai claims possiamo firmarlo con la nostra key, e inserire tutte le info che vogliamo



JWT – service gestione token



```
public String generaToken(Utente u){
    String ruolo=u.getRuolo().toString();
    String username=u.getEmail();
    String dataNascita=u.getDataDiNascita().format(DateTimeFormatter.ofPattern("EEEE dd MMMM yyyy"));
    String saluto="ciao, hai letto i miei dati, e adesso?";

    //1000L (1sec)*60(1 min)*60(1h)*24(1g)*60(60g)
    long millisecondiDiDurata=1000L*60*60*24*60;
    String token=Jwts.builder()
                    .claims()
                        .add("ruolo",ruolo)
                        .add("dataNascita",dataNascita)
                        .add("saluto",saluto)
                    //il subject di solito viene utilizzato per la convalida, ma come tutti è un tag opzionale
                        .subject(username)
                    //l'issuedAt è la data di creazione del token
                        .issuedAt(new Date(System.currentTimeMillis()))
                    //l'expiration è la data di scadenza del token
                        .expiration(new Date(System.currentTimeMillis()+millisecondiDiDurata))
                    .and()
            //la parte del signWith ci permette di "firmare" il nostro token e renderlo non modificabile
                    .signWith(getSignInKey())
             .compact();
    return token;
}
```



JWT – service gestione token - claims



- Tutti i dati inseriti saranno parte dei Claims, che posso ripendere dal token, se la firma del token è sbagliata o il token è scaduto verrò lanciata un'exception

```
private Claims estraiClaims(String token) {
    return (Claims)Jwts
            .parser()
                .verifyWith(getSignInKey())
            .build()
            .parse(token)
            .getPayload();
}
```



JWT – service gestione token - claims



- Una volta ottenuti i claims potremo prendere le info contenute come username e data di scadenza del token

```
private boolean isTokenScaduto(String token) {
    return getDataScadenza(token).before(new Date());
}

private Date getDataScadenza(String token) {
    return estraiClaims(token).getExpiration();
}

public String getUsername(String token) {
    return estraiClaims(token).getSubject();
}
```



JWT – service gestione token



- Ora finalmente potremmo sapere se il token è ancora valido controllando tutti i parametri dell'utente presi dal database

```
public boolean isTokenValid(String token) {
    String email=getUsername(token)
    Utente u=service.findByEmail(email);
    boolean b1= u.isUtenteAbilitato();
    boolean b2= !isTokenScaduto(token);
    boolean b3= !u.isScaduto();
    return b1&&b2&&b3;
}
```



JWT – bean di configurazione



- Altra parte fondamentale è una classe di configurazione annotata come `@Configuration` che ci permetterà di creare dei bean per configurare tutti gli oggetti di cui avremo bisogno per la nostra security, questa classe utilizzerà un oggetto per il findByEmail o findByUsername

```
@Configuration
@RequiredArgsConstructor
public class ConfigurationBean {

    private final UtenteService service;

}
```



JWT – bean di configurazione



- I bean contenuti in questa classe saranno 4, il primo si baserà sulla repository dell'utente e ci permetterà, partendo dallo username di prendere l'utente

```
@Bean
protected UserDetailsService userDetailsService() {
    return username -> service.findByEmail(username)
            .orElseThrow(
() -> new ResponseStatusException(HttpStatus.NOT_FOUND,"nessun utente nel sistema con
l'email selezionata")
        );
}
```

- Il secondo servirà per creare un PasswordEncoder,

```
@Bean
protected PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```



JWT – bean di configurazione



- Il terzo prenderà il manager di autenticazione dalle configurazioni di default

```
@Bean
protected AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws
Exception {
    return config.getAuthenticationManager();
}
```

- L'ultimo gestirà il provider di autenticazione utilizzando gli altri bean

```
@Bean
protected AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new
DaoAuthenticationProvider(userDetailsService());
    authProvider.setPasswordEncoder(passwordEncoder());
    return authProvider;
}
```



JWT – OncePerRequestFilter



- Il filter di autenticazione sarà il punto centrale della nostra implementazione
- In questa parte del codice controlleremo che:
  - il token non sia scaduto
  - che l'utente abbia realmente i permessi per accedere alla risorsa richiesta
- Qui avremo bisogno del service di gestione dei token e del service dello UserDetails
- Il metodo più importante è il doFilterInternal che farà l'intero controllo



JWT – OncePerRequestFilter



```
Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain chain) throws ServletException, IOException {
    String token=request.getHeader("Authorization");

    if(token!=null && token.startsWith("Bearer") {
         Utente user=gestoreToken.findByToken(token);
        if (SecurityContextHolder.getContext().getAuthentication() == null) {
            UsernamePasswordAuthenticationToken upat = new
UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
            upat.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(upat);
        }
    } chain.doFilter(request,response);
}
```



JWT – Configuratore Path



- L’ultima classe da creare sarà la classe di configuazione che conterrà il bean per la creazione della SecurityFilterChain
- Qui avremo bisogno del filter di autenticazione (OncePerRequestFilter) e dell’AuthenticationProvider
- Andremo a disabilitare le csrf e la session e abiliteremo i path che ci interessano solo agli utenti che hanno il permesso per quei path
- Oltre all’annotation `@Configuration` la classe avrà anche l’annotation `@EnableWebSecurity`



JWT – SecurityFilterChain 3.1



```java
@Configuration
@EnableWebSecurity
public class ConfiguratorePath {
    private final FilterAutenticazione jwtAuthFilter;
    private final AuthenticationProvider authenticationProvider;
    public ConfiguratorePath(FilterAutenticazione jwtAuthFilter, AuthenticationProvider authenticationProvider) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.authenticationProvider = authenticationProvider;
    }

    @Bean
    protected SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(
                        auth -> auth
                                .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                                .requestMatchers("/all/**").permitAll()
                                .requestMatchers("/admin/**").hasRole(GESTORE.getNome())
                                .requestMatchers("/studente/**").hasAnyRole(STUDENTE.getNome();GESTORE.getNome())
                                .anyRequest().authenticated()
                )
                .sessionManagement(sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .cors(AbstractHttpConfigurer::disable)
                .authenticationProvider(authenticationProvider)
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```



JWT – SecurityFilterChain 3.0



```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
                .csrf().disable()

        .authorizeHttpRequests().requestMatchers("/all/**").permitAll()
        .requestMatchers("/admin/**").hasRole(Ruolo.GESTORE.getNome())
                .requestMatchers("/user/**")

        .hasAnyRole(Ruolo.GESTORE.getNomeTrimmed(),Ruolo.UTENTE.getNome())
                .anyRequest().authenticated()
                .and()
                .sessionManagement()
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                .and()
                .authenticationProvider(authenticationProvider)
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
}
```



JWT – Gestire la login



- Nella login non torneremo più l'utente ma inseriremo come header il nostro token con la chiave «Authorization»

```
@PostMapping("/login")
public ResponseEntity<String> login(@RequestBody LoginRequest request)
{
    Utente u=utenteService.login(request);
    return ResponseEntity.status(HttpStatus.OK).header("Authorization",
gestoreToken.generaToken(u)).build();
}
```



JWT – Prendere l'utente nel metodo



- Per prendere l'utente negli hand point possiamo utilizzare come parametro d'ingresso annotato come @AuthenticationPrincipal
- Ovviamente il paramentro deve essere di un tipo di UserDetails, lo stesso caricato all'interno dello UsernamePasswordAuthenticationToken nel filter

```
 @GetMapping("/getUtente")
    public ResponseEntity<String>
prendiUtente(@AuthenticationPrincipal Utente u){

    }
}
```

